

American International University-Bangladesh (AIUB)
Department of computer science
Faculty of Science & Technology (FST)

Semester: FALL 2025-26
Course: Introduction to Data Science
Section : J
Group: 7

Project Title:
Video Game Sales Data Cleaning and Exploratory Data Analysis

Dataset Title: Video Games Sales

Supervised By:
Kamrun Naher Koli
Lecturer,
Department of Computer Science
Faculty of Science and Technology
American International University-Bangladesh

Submitted By :

Name	ID
APURBO, NAFIUN	23-50064-1
RAKIN, RAIYEN ZAYED	23-50071-1
SHEHORY, SABA MORIUM	23-50827-1

Submission Date : 06-04-2026

1. Introduction

This project focuses on data preprocessing, cleaning, and exploratory data analysis using the R programming language. The dataset used in this project contains video game sales data, including information such as platform, genre, publisher, critic scores, user scores, and global sales.

The main objective of this project is to:

- Identify and handle missing values
- Detect and remove outliers
- Normalize data
- Remove duplicate records
- Balance the dataset
- Perform descriptive statistical analysis

The project follows a structured pipeline to ensure the dataset is properly prepared for analysis.

2. Dataset Description

The dataset used in this project is titled **"Video Games Sales (as at 22 Dec 2016)"**, which contains comprehensive information about video game performance across different regions and platforms. It consists of multiple records, where each row represents a unique video game along with its associated attributes. The dataset includes a mix of numerical and categorical variables, allowing for a detailed analysis of sales trends, user engagement, and game characteristics.

Key Attributes

- Name – Game title
- Platform – Gaming platform
- Year – Release year
- Genre – Type of game
- Publisher – Publishing company
- NA_Sales, EU_Sales, JP_Sales – Regional sales
- Global_Sales – Total worldwide sales
- Critic_Score – Score given by critics
- User_Score – Score given by users
- Developer – Game developer
- Rating – Age rating

The dataset contains both numerical and categorical variables, making it suitable for preprocessing and analysis. Overall, this dataset provides a well-rounded view of video game performance and is appropriate for applying various data science techniques such as missing value handling, outlier detection, normalization, and statistical analysis.

3. Project Implementation

Task 1: Setup Libraries

Code

```
install.packages("dplyr")
install.packages("readr")
install.packages("tidyr")

library(dplyr)
library(readr)
library(tidyr)
```

Output

```
> library(dplyr)
> library(readr)
> library(tidyr)
```

Interpretation

These libraries are used for data manipulation, reading datasets, handling missing values, and visualization.

Data Import (readr): This library is utilized to load the Midterm.csv dataset into the R environment. It ensures that the raw survey data is imported correctly and efficiently.

Data Preparation (dplyr & tidyr): These packages are essential for organizing the data. dplyr is used to filter, select, and summarize specific rows, while tidyr handles the imputation of missing values to ensure the dataset is complete before analysis.

Task 2: Load and Inspect Data

Code

```
vg_data <- read_csv("D:/AIUB Notes/Introduction to Data Science/main/Video_Games_Sales_as_at_22_Dec_2016 2.csv")
vg_data <- vg_data %>% filter(!is.na(Name))
# Rename columns for cleaner access
new_colnames <- c("Name", "Platform", "Year", "Genre", "Publisher", "NA_Sales", "EU_Sales", "JP_Sales", "Other_Sales",
                  "Global_Sales", "Critic_Score", "Critic_Count",
                  "User_Score", "User_Count", "Developer", "Rating")
colnames(vg_data) <- new_colnames

# Data Inspection
dim(vg_data)
head(vg_data)
str(vg_data)
summary(vg_data)
```

Output

```
> vg_data <- read_csv("D:/AIUB Notes/Introduction to Data Science/main/Video_Games_Sales_as_at_22_Dec_2016 2.csv")
Rows: 16719 Columns: 16
# Column specification
Delimiter: ","
chr (8): Name, Platform, Year_of_Release, Genre, Publisher, User_Score, Developer, Rating
dbl (8): NA_Sales, EU_Sales, JP_Sales, Other_Sales, Global_Sales, Critic_Score, Critic_Count, User_Count

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.
> vg_data <- vg_data %>% filter(!is.na(Name))
> # Rename columns for cleaner access
> new_colnames <- c("Name", "Platform", "Year", "Genre", "Publisher", "NA_Sales", "EU_Sales", "JP_Sales", "Other_Sales",
+                  "Global_Sales", "Critic_Score", "Critic_Count",
+                  "User_Score", "User_Count", "Developer", "Rating")
> colnames(vg_data) <- new_colnames
> # Data Inspection
> dim(vg_data)
[1] 16717 16
> head(vg_data)
# A tibble: 6 x 16
   Name Platform Year Genre Publisher NA_Sales EU_Sales JP_Sales Other_Sales Global_Sales Critic_Score Critic_Count User_Score
   <chr> <chr> <chr> <chr> <chr> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <dbl> <chr>
1 Wii   Wii    2006 Spor... Nintendo 41.4 29.0 3.77 8.45 82.5 76 51 8
2 Supe... NES   1985 Plat... Nintendo 29.1 3.58 6.81 0.77 40.2 NA NA NA
3 Mari... Wii    2008 Rac... Nintendo 15.7 12.8 3.79 3.29 35.5 82 73 8.3
4 Wii   Wii    2009 Spor... Nintendo 15.6 10.9 3.28 2.95 32.8 80 73 8
5 Poke... GB    1996 Role... Nintendo 11.3 8.89 10.2 1 31.4 NA NA NA
6 Tetr... GB    1989 Puzz... Nintendo 23.2 2.26 4.22 0.58 30.3 NA NA NA

# i 3 more variables: User_Count <dbl>, Developer <chr>, Rating <chr>

> str(vg_data)
spec_tbl_ [16,717 x 16] (S3: spec_tbl_df/tbl_df/tbl/data.frame)
 $ Name      : chr [1:16717] "Wii Sports" "Super Mario Bros." "Mario Kart Wii" "Wii Sports Resort" ...
 $ Platform  : chr [1:16717] "Wii" "NES" "Wii" "Wii" ...
 $ Year      : chr [1:16717] "2006" "1985" "2008" "2009" ...
 $ Genre     : chr [1:16717] "Sports" "Platform" "Racing" "Sports" ...
 $ Publisher : chr [1:16717] "Nintendo" "Nintendo" "Nintendo" "Nintendo" ...
 $ NA_Sales  : num [1:16717] 41.4 29.1 15.7 15.6 11.3 ...
 $ EU_Sales  : num [1:16717] 28.96 3.58 12.76 10.93 8.89 ...
 $ JP_Sales  : num [1:16717] 3.77 6.81 3.79 3.28 10.22 ...
 $ Other_Sales : num [1:16717] 8.45 0.77 3.29 2.95 1 0.58 2.88 2.84 2.24 0.47 ...
 $ Global_Sales: num [1:16717] 82.5 40.2 35.5 32.8 31.4 ...
 $ Critic_Score: num [1:16717] 76 NA 82 80 NA NA 89 58 87 NA ...
 $ Critic_Count: num [1:16717] 51 NA 73 73 NA NA 65 41 80 NA ...
 $ User_Score  : chr [1:16717] "8" NA "8.3" "8" ...
 $ User_Count  : num [1:16717] 322 NA 709 192 NA NA 431 129 594 NA ...
 $ Developer   : chr [1:16717] "Nintendo" NA "Nintendo" "Nintendo" ...
 $ Rating      : chr [1:16717] "E" NA "E" "E" ...
- attr(*, "spec")=
  .. cols(
  ..   Name = col_character(),
  ..   Platform = col_character(),
  ..   Year_of_Release = col_character(),
  ..   Genre = col_character(),
  ..   Publisher = col_character(),
  ..   NA_Sales = col_double(),
  ..   EU_Sales = col_double(),
  ..   JP_Sales = col_double(),
  ..   Other_Sales = col_double(),
  ..   Global_Sales = col_double(),
  ..   Critic_Score = col_double(),
  ..   Critic_Count = col_double(),
  ..   User_Score = col_character(),
  ..   User_Count = col_double(),
  ..   Developer = col_character(),
  ..   Rating = col_character()
  .. )
- attr(*, "problems")=<externalptr>
```

Interpretation

The dataset is loaded and inspected to understand its structure, dimensions, and summary statistics.

Task 3: Identify Missing Value

Code

```
cat("\n[Task 3] Checking for missing values...\n")
working_data <- vg_data # working copy

na_counts <- colSums(is.na(working_data))
print(na_counts)

total_na <- sum(na_counts)
cat("Total missing cells:", total_na, "\n")

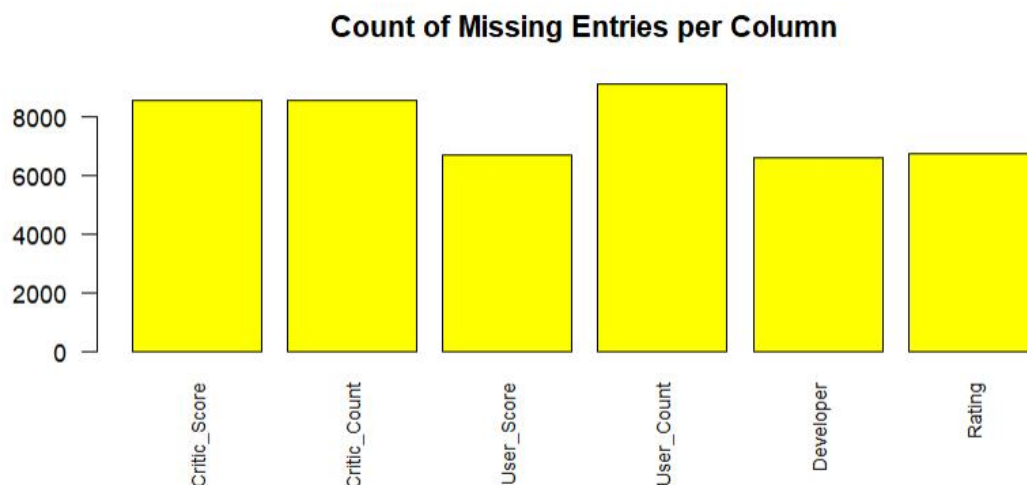
# Visualization: Missing Data
par(mar = c(8, 5, 4, 2))
barplot(na_counts[na_counts > 0],
        main = "Count of Missing Entries per Column",
        col = "yellow",
        las = 2,
        cex.names = 0.8)
```

Output

```
> cat("\n[Task 3] Checking for missing values...\n")

[Task 3] Checking for missing values...
> working_data <- vg_data # working copy
> na_counts <- colSums(is.na(working_data))
> print(na_counts)
      Name      Platform      Year      Genre      Publisher      NA_Sales      EU_Sales      JP_Sales      Other_Sales
Global_Sales Critic_Score Critic_Count User_Score User_Count Developer      Rating
      0           8580           8580        6702        9127        6621        6767
> total_na <- sum(na_counts)
> cat("Total missing cells:", total_na, "\n")
Total missing cells: 46377
> # Visualization: Missing Data
> par(mar = c(8, 5, 4, 2))
> barplot(na_counts[na_counts > 0],
+         main = "Count of Missing Entries per Column",
+         col = "darkcyan",
+         las = 2,
+         cex.names = 0.8)
```

Barplot



Interpretation

Missing values are identified across all columns. This helps determine which attributes require cleaning.

Task 4: Handling Missing Values

Code

```
calc_mode <- function(v) {
  uniqv <- unique(na.omit(v))
  if (length(uniqv) == 0) return(NA)
  uniqv[which.max(tabulate(match(v, uniqv)))]
}

# Numeric columns → median
avg_year <- median(working_data$Year, na.rm = TRUE)
avg_critic_score <- median(working_data$Critic_Score, na.rm = TRUE)
avg_critic_count <- median(working_data$Critic_Count, na.rm = TRUE)
avg_user_score <- median(working_data$User_Score, na.rm = TRUE)
avg_user_count <- median(working_data$User_Count, na.rm = TRUE)

# Categorical columns → mode
mode_genre <- calc_mode(working_data$Genre)
mode_publisher <- calc_mode(working_data$Publisher)
mode_developer <- calc_mode(working_data$Developer)
mode_rating <- calc_mode(working_data$Rating)

# Apply Imputation
processed_data <- working_data %>%
  mutate(
    Year = replace_na(Year, avg_year),
    Critic_Score = replace_na(Critic_Score, avg_critic_score),
    Critic_Count = replace_na(Critic_Count, avg_critic_count),
    User_Score = replace_na(User_Score, avg_user_score),
    User_Count = replace_na(User_Count, avg_user_count),

    Genre = replace_na(Genre, mode_genre),
    Publisher = replace_na(Publisher, mode_publisher),
    Developer = replace_na(Developer, mode_developer),
    Rating = replace_na(Rating, mode_rating)
  )

# Verify cleanup
cat("\n[Check] Remaining NAs after imputation:\n")
print(colSums(is.na(processed_data)))

# Create binary target column: High_Seller (1 = above median Global Sales)
median_sales <- median(processed_data$Global_Sales, na.rm = TRUE)
processed_data <- processed_data %>%
  mutate(High_Seller = ifelse(Global_Sales > median_sales, 1, 0))
```

Output

```
> calc_mode <- function(v) {
+   uniqv <- unique(na.omit(v))
+   if (length(uniqv) == 0) return(NA)
+   uniqv[which.max(tabulate(match(v, uniqv)))]
+ }
> # Numeric columns + median
> avg_year <- median(working_data$Year, na.rm = TRUE)
> avg_critic_score <- median(working_data$Critic_Score, na.rm = TRUE)
> avg_critic_count <- median(working_data$Critic_Count, na.rm = TRUE)
> avg_user_score <- median(working_data$User_Score, na.rm = TRUE)
> avg_user_count <- median(working_data$User_Count, na.rm = TRUE)
> # Categorical columns + mode
> mode_genre <- calc_mode(working_data$Genre)
> mode_publisher <- calc_mode(working_data$Publisher)
> mode_developer <- calc_mode(working_data$Developer)
> mode_rating <- calc_mode(working_data$Rating)
> # Apply Imputation
> processed_data <- working_data %>%
+   mutate(
+     Year = replace_na(Year, avg_year),
+     Critic_Score = replace_na(Critic_Score, avg_critic_score),
+     Critic_Count = replace_na(Critic_Count, avg_critic_count),
+     User_Score = replace_na(User_Score, avg_user_score),
+     User_Count = replace_na(User_Count, avg_user_count),
+
+     Genre = replace_na(Genre, mode_genre),
+     Publisher = replace_na(Publisher, mode_publisher),
+     Developer = replace_na(Developer, mode_developer),
+     Rating = replace_na(Rating, mode_rating)
+   )
> # Verify cleanup
> cat("\n[Check] Remaining NAs after imputation:\n")

[Check] Remaining NAs after imputation:
> print(colSums(is.na(processed_data)))
      Name      Platform      Year      Genre      Publisher      NA_Sales      EU_Sales
      0           0           0           0           0           0           0
      JP_Sales  Other_Sales  Global_Sales  Critic_Score  Critic_Count  User_Score  User_Count
      0           0           0           0           0           0           0
      Developer      Rating
      0           0
```

```
> # Create binary target column: High_Seller (1 = above median Global Sales)
> median_sales <- median(processed_data$Global_Sales, na.rm = TRUE)
> processed_data <- processed_data %>%
+   mutate(High_Seller = ifelse(Global_Sales > median_sales, 1, 0))
```

Interpretation

This ensures that missing values are replaced with statistically appropriate values, preserving data integrity.

Task 5: Outlier Detection and Removal

Code

```
# IQR Filtering
quantiles <- quantile(processed_data$Global_Sales, probs = c(0.25, 0.75), na.rm = TRUE)
iqr_value <- IQR(processed_data$Global_Sales, na.rm = TRUE)
lower_bound <- quantiles[1] - 1.5 * iqr_value
upper_bound <- quantiles[2] + 1.5 * iqr_value

processed_data <- processed_data %>%
  filter(Global_Sales >= lower_bound & Global_Sales <= upper_bound)

cat("\nRows remaining after outlier removal:", nrow(processed_data), "\n")
```

Output

```
> # IQR Filtering
> quantiles <- quantile(processed_data$Global_Sales, probs = c(0.25, 0.75), na.rm = TRUE)
> iqr_value <- IQR(processed_data$Global_Sales, na.rm = TRUE)
> lower_bound <- quantiles[1] - 1.5 * iqr_value
> upper_bound <- quantiles[2] + 1.5 * iqr_value
> processed_data <- processed_data %>%
+   filter(Global_Sales >= lower_bound & Global_Sales <= upper_bound)
> cat("\nRows remaining after outlier removal:", nrow(processed_data), "\n")
```

Rows remaining after outlier removal: 14826

Interpretation

The IQR method is used to remove extreme values from the dataset, improving accuracy and consistency.

Task 6: Remove Duplicates and Normalize

Code

```
cat("\n[Task 6] Removing Duplicates...\n")
initial_rows <- nrow(processed_data)
processed_data <- distinct(processed_data)
cat("Rows dropped:", initial_rows - nrow(processed_data), "\n")

# Min-Max Normalize Critic Score
min_max_norm <- function(x) {
  (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
}

processed_data$Critic_Score_Norm <- min_max_norm(processed_data$Critic_Score)
cat("\nCritic Score Norm summary:\n")
print(summary(processed_data$Critic_Score_Norm))
```

Output

```
> cat("\n[Task 6] Removing Duplicates...\n")

[Task 6] Removing Duplicates...
> initial_rows <- nrow(processed_data)
> processed_data <- distinct(processed_data)
> cat("Rows dropped:", initial_rows - nrow(processed_data), "\n")
Rows dropped: 0
> # Min-Max Normalize Critic Score
> min_max_norm <- function(x) {
+   (x - min(x, na.rm = TRUE)) / (max(x, na.rm = TRUE) - min(x, na.rm = TRUE))
+ }
> processed_data$Critic_Score_Norm <- min_max_norm(processed_data$Critic_Score)
> cat("\nCritic Score Norm summary:\n")

Critic Score Norm summary:
> print(summary(processed_data$Critic_Score_Norm))
   Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
0.0000 0.6706 0.6824 0.6608 0.6824 1.0000
```

Interpretation

Duplicate rows are removed, and normalization is applied to scale numerical values between 0 and 1.

Task 7: Data Balancing and Splitting

Code

```

cat("\n[Task 7] Balancing Data...\n")
target_counts <- table(processed_data$High_Seller)
min_class_size <- min(target_counts)

balanced_data <- processed_data %>%
  group_by(High_Seller) %>%
  sample_n(min_class_size) %>%
  ungroup()

print(table(balanced_data$High_Seller))

# 80/20 Train-Test Split
set.seed(123)
balanced_data$row_id <- 1:nrow(balanced_data)
train_df <- balanced_data %>% sample_frac(0.8)
test_df <- anti_join(balanced_data, train_df, by = "row_id") %>% select(-row_id)
train_df <- train_df %>% select(-row_id)

cat("\nTraining set rows:", nrow(train_df), "\n")
cat("Test set rows: ", nrow(test_df), "\n")

```

Output

```

> cat("\n[Task 7] Balancing Data...\n")

[Task 7] Balancing Data...
> target_counts <- table(processed_data$High_Seller)
> min_class_size <- min(target_counts)
> balanced_data <- processed_data %>%
+   group_by(High_Seller) %>%
+   sample_n(min_class_size) %>%
+   ungroup()
> print(table(balanced_data$High_Seller))

  0    1
6310 6310
> # 80/20 Train-Test Split
> set.seed(123)
> balanced_data$row_id <- 1:nrow(balanced_data)
> train_df <- balanced_data %>% sample_frac(0.8)
> test_df <- anti_join(balanced_data, train_df, by = "row_id") %>% select(-row_id)
> train_df <- train_df %>% select(-row_id)
> cat("\nTraining set rows:", nrow(train_df), "\n")

Training set rows: 10096
> cat("Test set rows: ", nrow(test_df), "\n")
Test set rows:      2524

```

Interpretation

The dataset is balanced using sampling and split into training (80%) and testing (20%) sets.

Task 8: Processed data checking and saving

Code

```

cat("\n[Done] Preprocessed data is ready.\n")
cat("Final columns:\n")
print(colnames(processed_data))
write_csv(processed_data, "VG_Processed.csv")

```

Output

```

> cat("Final columns:\n")
Final columns:
> print(colnames(processed_data))
 [1] "Name"           "Platform"      "Year"          "Genre"
 [5] "Publisher"      "NA_Sales"      "EU_Sales"      "JP_Sales"
 [9] "Other_Sales"    "Global_Sales"  "Critic_Score"  "Critic_Count"
[13] "User_Score"     "User_Count"    "Developer"     "Rating"
[17] "High_Seller"    "Critic_Score_Norm"
> write_csv(processed_data, "VG_Processed.csv")

```